TAKE-HOME

Prueba técnica — Tech Lead

Hola, gracias por llegar hasta acá.

- Tienes una ventana de **5 días** para entregarlo. Si te toma mucho más, para y entrega lo que tengas con una nota de qué faltó y por qué: eso también nos dice algo útil.
- **Lenguaje:** elige **Go**, **Java (Spring Boot / WebFlux)** o **NestJS**.
- **Plus opcional:** si tienes soltura en front, una pantalla en Angular o Flutter suma, pero no penaliza no hacerla. Está descrita al final.

Contexto del dominio

Operamos un marketplace B2B donde tenderos compran a distribuidores de bebidas. La app corre en varios países (Colombia, Ecuador, Guatemala, Perú, entre otros) y varias marcas comerciales, sobre la misma plataforma multi-tenant.

Un pedido pasa por:

catálogo → carrito → checkout (cotización) → confirmación → historial

Sobre el precio actúan varias palancas comerciales que **pueden coexistir en el mismo pedido**:

PALANCA	REGLA
Descuento por escala	A partir de N unidades de un SKU o familia, se aplica un % sobre el valor de esas unidades. Las escalas son acumulativas por tramos.
Combo	Comprar un conjunto de SKU en cantidades mínimas da un precio de combo sobre ese conjunto.
Obsequio	Al cumplir una condición de compra, se agregan unidades gratis de un SKU. El obsequio no paga producto, pero sí puede tener implicaciones tributarias.
Impuestos	Cada SKU tiene una tasa. El impuesto se calcula sobre la base gravable después de descuentos comerciales.
Cupo de crédito	Un cliente con crédito tiene un cupo disponible. Un pedido a crédito solo procede si el total cabe en el cupo, y el cupo debe reflejar el consumo.

Parte A · Take-home

A1 — Motor de cálculo del pedido

Implementa el cálculo del total de un pedido dado un carrito y un conjunto de palancas vigentes.

REQUISITOS FUNCIONALES

- Un endpoint (o caso de uso, si prefieres no exponer HTTP) que reciba el carrito y devuelva el desglose: **por línea y totales** — subtotal bruto, descuentos aplicados con su origen, obsequios con su cantidad, base gravable, impuestos y total a pagar.
- Las palancas pueden coexistir. **Define y documenta el orden de aplicación** y por qué ese orden es el correcto.
- Si el cliente paga a crédito, el cálculo debe indicar si el pedido cabe en el cupo disponible.

REQUISITOS NO FUNCIONALES — ESTOS SON EL CORAZÓN DEL EJERCICIO

- **El total cotizado y el total confirmado deben ser idénticos.** Diseña para que no puedan divergir, aunque las palancas o los precios cambien entre un momento y otro. Explica tu mecanismo.
- **Nada de aritmética de punto flotante para dinero.** Sé explícito con el tipo, la escala decimal y la regla de redondeo, y con dónde se redondea (¿por línea? ¿por total? ¿por impuesto?).
- **Multi-país:** la moneda, la escala decimal y las tasas no pueden estar quemadas en la lógica.

COBERTURA DE PRUEBAS — OBLIGATORIA

Entrega **golden tests** que cubran, como mínimo:

1. Un pedido con descuento en dos tramos.
2. Un pedido con combo donde los productos que activan el combo no deben aparecer como un descuento suelto.
3. Un pedido con obsequio: el producto gratis aparece con su cantidad en el desglose.
4. Un combo con impuestos: verifica que el impuesto entra en el total.
5. Un pedido a crédito que excede el cupo.
6. El caso de regresión de la cotización: mismo carrito, cotizo y confirmo, los dos totales coinciden.

A2 — Confirmación del pedido: idempotencia y consistencia

> **Nos pasó, en tres releases distintas,** que un pedido se confirmaba, le devolvíamos número de orden al tendero, y después no aparecía en su historial — en algunos casos ni siquiera había quedado registrado.

Diseña e implementa la confirmación del pedido de forma que eso sea **imposible**.

- `POST /orders` acepta una **clave de idempotencia**. Reintentos del cliente (mala red, doble tap, reintento de la app) no pueden crear pedidos duplicados ni consumir el cupo dos veces.
- Si la respuesta es **201 con número de orden**, la orden existe y es consultable. Define qué garantiza esa invariante.
- La confirmación dispara **efectos hacia afuera** — al menos: notificación push al tendero y publicación del pedido para el ERP. Explica cómo garantizas que esos efectos no se pierdan ni se dupliquen, y qué pasa si el sistema externo está caído.
- Documenta el **modelo de consistencia** que elegiste y qué se sacrifica.

Puedes usar la base de datos que quieras (o una en memoria con una interfaz bien definida). No necesitamos infraestructura desplegada: nos interesa el diseño y que los tests demuestren la invariante.

A3 — ADR (máximo 1 página)

Un documento corto, del tipo que le pasarías a tu equipo:

- Decisiones que tomaste y las alternativas que descartaste.
- Qué **no** implementaste conscientemente y por qué.
- Qué revisarías en un code review de este mismo código si lo hubiera escrito alguien más.
- Un riesgo que ves en el diseño y cómo lo mitigarías.

Entrega

Repositorio Git (público, o privado con acceso) con:

- El código y los tests, con **un comando único** para correrlos.
- Un **README** con: cómo correrlo, el orden de aplicación de palancas, la política de redondeo y el modelo de consistencia.
- El **ADR**.
- **Historial de commits legible.** No hace falta que sea perfecto; sí que se entienda cómo avanzaste.

Plus opcional · Front

Si quieres mostrar tu lado de front, elige una de estas dos y dedícale como mucho **una hora**:

- **Angular** o **Flutter** — pantalla de resumen del pedido. Consume tu propio cálculo (o un JSON fijo) y muestra el desglose: líneas, descuentos con su origen, obsequios con cantidad, impuestos y total.

Nos fijaremos en tres cosas, porque son defectos que tuvimos:

1. **Ningún texto se corta.** Pruébalo con nombres de producto largos y en una pantalla mediana, no solo en tu dispositivo.
2. **Ningún control queda tapado** por la barra de navegación del sistema o por el teclado.
3. **Las imágenes de un mismo grid escalan con el mismo criterio.**

Si además nos cuentas cómo evitarías que esos tres defectos se repitan **a nivel de componente y no de pantalla**, mejor.
